

RESOURCE MANAGEMENT

SCENARIO 1 — No Requests, No Limits → Node OOM Risk

“This pod has no limits. It will try to consume more than node capacity (2GB).
Kubernetes will NOT stop it. Linux will step in.”

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: no-limit-oom
spec:
  containers:
    - name: stress
      image: polinux/stress
      command: ["stress"]
      args: ["--vm", "1", "--vm-bytes", "3G", "--vm-hang", "1"]
```

What happens

- Pod tries → **3GB**
- Node has → **2GB**

 No cgroup limit → it consumes freely

Result

- Node memory exhausted
 - Linux triggers OOM killer
 - One process gets killed
-

What to show

kubectl describe pod no-limit-oom

Possible:

Reason: OOMKilled

“Without limits, Kubernetes cannot protect the node. Linux decides who dies.”

SCENARIO 2 — Requests Only → Scheduler Control, Still Dangerous

“This pod requests 500MB, but will try to use 2GB.

Scheduler allows it, but runtime is uncontrolled.”

YAML

apiVersion: v1

kind: Pod

metadata:

name: request-only-overuse

spec:

containers:

- name: stress

image: polinux/stress

command: ["stress"]

args: ["--vm", "1", "--vm-bytes", "2G", "--vm-hang", "1"]

resources:

requests:

memory: "500Mi"

cpu: "200m"

What happens

- Scheduler sees → 500Mi → OK
 - Pod starts
 - Actually uses → 2GB
-

Result

- 👉 Node memory pressure again
 - 👉 Linux OOM killer OR kubelet eviction
-

“Requests control placement, not usage.”

🔧 **SCENARIO 3 — Requests + Limits → Controlled OOM (Container-Level)**

🎤 **What to say**

“Now I enforce limits. The container cannot cross 1GB.”

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: limit-enforced-oom
spec:
  containers:
    - name: stress
      image: polinux/stress
      command: ["stress"]
      args: ["--vm", "1", "--vm-bytes", "1500M", "--vm-hang", "1"]
      resources:
        requests:
          memory: "500Mi"
          cpu: "200m"
        limits:
          memory: "1Gi"
          cpu: "500m"
```

⚙️ **What happens**

- Pod tries → 1.5GB
 - Limit = 1GB
-

🌟 **Result**

- 👉 cgroup enforces limit
 - 👉 container killed (OOMKill)
-

Show

kubectl get pod limit-enforced-oom

RESTARTS increasing

“This is container-level OOM, controlled by Kubernetes via Linux.”

SCENARIO 4 — Within Limit → Stable Pod

What to say

“Now the pod stays within limits. No crash.”

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: safe-pod
spec:
  containers:
    - name: stress
      image: polinux/stress
      command: ["stress"]
      args: ["--vm", "1", "--vm-bytes", "300M", "--vm-hang", "1"]
      resources:
        requests:
          memory: "200Mi"
          cpu: "100m"
        limits:
          memory: "500Mi"
          cpu: "300m"
```

What happens

- Uses 300MB
- Limit = 500MB

🌟 Result

- 👉 No OOM
- 👉 Stable pod

“Correct sizing prevents failures.”

🧪 SCENARIO 5 — Node Memory Pressure + Eviction

Now we trigger **real eviction**.

“Now I will overload the node using multiple pods.
Kubernetes will start evicting pods to protect the node.”

YAML (BestEffort Pod)

```
apiVersion: v1
kind: Pod
metadata:
  name: besteffort-pod
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "700M", "--vm-hang", "1"]
```

YAML (Burstable Pod)

```
apiVersion: v1
kind: Pod
metadata:
  name: burstable-pod
```

```
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "700M", "--vm-hang", "1"]
    resources:
      requests:
        memory: "200Mi"
        cpu: "100m"
      limits:
        memory: "800Mi"
        cpu: "300m"
```

YAML (Guaranteed Pod)

```
apiVersion: v1
kind: Pod
metadata:
  name: guaranteed-pod
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "700M", "--vm-hang", "1"]
    resources:
      requests:
        memory: "700Mi"
        cpu: "200m"
      limits:
        memory: "700Mi"
        cpu: "200m"
```

How to run

Apply all 3:

```
kubectl apply -f ...
```

⚙️ What happens

Total memory demand:

$700 + 700 + 700 = 2.1\text{GB}$

Node = 2GB

🌟 Eviction order

Kubernetes kills:

1. BestEffort
 2. Burstable
 3. Guaranteed (last)
-

👁️ What to show

`kubectl get pods`

You will see:

Evicted

“Kubernetes sacrifices low-priority pods to save the node.”

“QoS determines survival priority.”

When memory exceeds node capacity:

Step-by-step

1. Memory usage increases
 2. kubelet detects pressure
 3. Eviction starts (QoS based)
 4. If too fast → Linux OOM killer
-

Will node crash?

❌ Normally NO

Because:

- kubelet evicts pods
 - Linux kills processes
-

! Node crashes only if:

- system processes killed
- kernel panic
- extreme overload

ResourceQuota (Namespace-Level Control)

“Now we’ll create a namespace with a quota:

Max 2 pods, max 1 GB total memory requested, 1.5 GB total memory limit.”

“Any attempt to create more than that will fail.”

YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: quota-demo
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: quota
  namespace: quota-demo
spec:
  hard:
    pods: "2"
    requests.memory: "1Gi"
    limits.memory: "1.5Gi"
```

What happens

- Kubernetes tracks **total resource usage in this namespace**.
 - If a user tries to create **extra pods or pods exceeding total request/limit**, it fails.
-

Live demo YAML to test quota

```
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  namespace: quota-demo
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "700M", "--vm-hang", "1"]
  resources:
    requests:
      memory: "500Mi"
    limits:
      memory: "800Mi"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod2
  namespace: quota-demo
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "500M", "--vm-hang", "1"]
  resources:
    requests:
      memory: "600Mi"
    limits:
      memory: "800Mi"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod3
  namespace: quota-demo
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "400M", "--vm-hang", "1"]
  resources:
    requests:
      memory: "400Mi"
    limits:
      memory: "500Mi"
```

💥 What happens

- Pod1 → allowed (500Mi request)
- Pod2 → allowed (600Mi request)

Total request now = 1.1 GB > quota 1 GB → pod2 fails if we create sequentially.

- Pod3 → fails due to **pod count limit**
-

👁️ What to show

```
kubectl get pods -n quota-demo
```

```
kubectl describe pod pod2 -n quota-demo
```

- Shows: exceeded quota: pods or requests.memory
-

“ResourceQuota enforces **namespace-level limits**, not individual pod limits.”

LimitRange (Default / Min / Max per Pod)

“Developers often forget to set requests and limits.

LimitRange ensures every pod has defaults and enforces min/max.”

YAML — Namespace & LimitRange

```
apiVersion: v1
kind: Namespace
metadata:
  name: limitrange-demo
---
apiVersion: v1
kind: LimitRange
metadata:
  name: limit-range
  namespace: limitrange-demo
spec:
  limits:
    - type: Container
      default:
        memory: "300Mi"
        cpu: "200m"
      defaultRequest:
        memory: "200Mi"
        cpu: "100m"
      min:
        memory: "100Mi"
        cpu: "50m"
      max:
        memory: "500Mi"
        cpu: "500m"
```

What happens

- Any pod without resource requests → gets **defaultRequest** / **default limit**
- Any pod trying **less than min** or **more than max** → **rejected**

Live demo YAML to test LimitRange

Pod with NO resource defined (gets default)

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: default-pod
  namespace: limitrange-demo
spec:
  containers:
  - name: nginx
    image: nginx
```

Pod exceeding max → REJECTED

```
apiVersion: v1
kind: Pod
metadata:
  name: too-big-pod
  namespace: limitrange-demo
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "1G", "--vm-hang", "1"]
    resources:
      requests:
        memory: "600Mi"
      limits:
        memory: "1G"
```

Pod below min → REJECTED

```
apiVersion: v1
kind: Pod
metadata:
  name: too-small-pod
  namespace: limitrange-demo
spec:
  containers:
  - name: stress
    image: polinux/stress
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "50M", "--vm-hang", "1"]
    resources:
      requests:
        memory: "20Mi"
```

limits:
memory: "50Mi"

What happens

- default-pod → accepted, gets memory 200Mi request / 300Mi limit automatically
- too-big-pod → rejected
- too-small-pod → rejected